

Disueño

Patrones.

Nota: NO OLVIDAR, EN LAS TABLAS UML DEBEN IR LAS FLECHAS QUE INDICAN QUÉ HACE LA RELACIÓN. Ejemplo: Persona ->(renueva) Producto.

CLASIFICACIÓN DE LOS PATRONES DE DISEÑO

		Propósito		
		Creacional	Estructural	De comportamiento
Alcance	Clase	Factory Method	Adapter	Interpreter Template Method
	Objeto	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Facade Proxy	Chain of Responsibility Command Iterator Mediator Memento Flyweight (195) Observer State Strategy Visitor

Creacionales:

1. Factory Method

Qué es:

Define una interfaz para crear un objeto, pero permite que las subclases decidan qué clase concreta instanciar.

Ejemplo:

Imagina un sistema de notificaciones. Puedes tener una clase abstracta **Notificacion** y subclases como **EmailNotificacion** o **SMSNotificacion**.

El método **crearNotificacion()** decide en tiempo de ejecución qué tipo de notificación crear.

Cuándo usarlo:

Cuando necesitas crear objetos sin especificar exactamente qué clase usar. Ideal para frameworks o librerías extensibles.

2. Abstract Factory

Qué es:

Permite crear familias de objetos relacionados (por ejemplo, botones y menús para distintos sistemas operativos) sin especificar sus clases concretas.

Ejemplo:

Un programa multiplataforma puede tener una fábrica `GUIFactory` con implementaciones concretas `WindowsFactory` y `MacFactory`, que crean `BotonWindows` o `BotonMac`.

Cuándo usarlo:

Cuando el sistema debe ser independiente de cómo se crean los objetos o cuando los objetos deben ser compatibles entre sí (por ejemplo, interfaces coherentes entre componentes visuales).

3. Builder

Qué es:

Separa la construcción de un objeto complejo de su representación, permitiendo construirlo paso a paso.

Ejemplo:

Para construir una `Casa`, el `Builder` define pasos como `construirParedes()`, `construirTecho()`, `instalarPuertas()`. Así puedes crear casas de distintos tipos reutilizando la misma secuencia.

Cuándo usarlo:

Cuando un objeto requiere muchos pasos o configuraciones opcionales (por ejemplo, al crear documentos PDF, autos o configuraciones de software).

4. Prototype

Qué es:

Permite copiar objetos existentes sin conocer su estructura interna. La clonación se realiza a través de una interfaz `clone()` implementada por los propios objetos.

Ejemplo:

Si tienes un objeto `Enemigo` con muchas configuraciones, puedes clonar uno existente para crear un nuevo enemigo con ligeras variaciones (sin reconstruirlo desde cero).

Cuándo usarlo:

Cuando crear un objeto es costoso (por ejemplo, por cálculos complejos o lectura desde una base de datos), o cuando necesitas duplicar estructuras similares.

5. Singleton

Qué es:

Garantiza que una clase tenga solo una instancia y proporciona un punto de acceso global a ella.

Ejemplo:

Una clase **Logger** o **Configuracion** del sistema: solo debe haber una instancia en toda la aplicación que maneje los registros o configuraciones globales.

Cuándo usarlo:

Cuando necesitas controlar el acceso a un recurso compartido (como una base de datos, registro de logs, o configuración del sistema).

Resumen general

Patrón	Problema que resuelve	Ejemplo típico	Momento ideal de uso
Factory Method	Delegar la creación de objetos a subclases	Notificaciones (SMS, email)	Cuando hay múltiples variantes de un producto
Abstract Factory	Crear familias de objetos relacionados	Interfaces GUI multiplataforma	Cuando los productos deben ser coherentes entre sí
Builder	Construir objetos paso a paso	Construcción de casas, autos, informes	Cuando los objetos son complejos o tienen muchas configuraciones
Prototype	Copiar objetos sin conocer su estructura	Clonación de enemigos o documentos	Cuando crear desde cero es costoso
Singleton	Asegurar una única instancia global	Logger, conexión a base de datos	Cuando debe existir un solo punto de acceso

Estructurales

PATRONES DE DISEÑO ESTRUCTURALES

Estos patrones se centran en **cómo las clases y objetos se combinan para formar estructuras más grandes y flexibles**, sin perder eficiencia o claridad.

1. Adapter

Qué es:

Permite que dos clases con interfaces incompatibles trabajen juntas mediante un *adaptador* que traduce las llamadas entre ellas.

Ejemplo:

Un programa que usa una interfaz **USB** necesita conectarse a un dispositivo que solo tiene **HDMI**. El adaptador convierte las señales de un tipo al otro.

Cuándo usarlo:

Cuando quieres **reutilizar una clase existente** pero su interfaz no es compatible con la que necesitas (por ejemplo, integrar librerías externas).

2. Decorator

Qué es:

Permite agregar o quitar dinámicamente funcionalidades a objetos específicos, sin modificar su clase original.

Ejemplo:

En una aplicación de gráficos, puedes tener una clase **Ventana** y decorarla con **BordeDecorador** o **SombraDecorador**, añadiendo nuevas funciones visuales.

Cuándo usarlo:

Cuando necesitas **agregar comportamientos de forma flexible en tiempo de ejecución**, evitando crear múltiples subclases con combinaciones de funcionalidades.

3. Facade

Qué es:

Proporciona una **interfaz simplificada** para un sistema complejo, actuando como un punto de acceso único a varias clases o librerías.

Ejemplo:

Un sistema bancario con muchos módulos (cuentas, préstamos, transferencias) puede tener una clase **BancoFacade** que simplifica las operaciones para el usuario final.

Cuándo usarlo:

Cuando un sistema tiene **muchas clases o subsistemas difíciles de usar directamente** y se quiere ofrecer una interfaz unificada o más sencilla.

4. Proxy

Qué es:

Controla el acceso a un objeto, permitiendo ejecutar código adicional antes o después de invocar sus métodos. Puede servir como sustituto temporal del objeto real.

Ejemplo:

Un **ProxyImagen** carga la imagen solo cuando se necesita (carga diferida), o un **ProxySeguro** valida permisos antes de acceder a un servicio.

Cuándo usarlo:

Cuando deseas **controlar el acceso, mejorar el rendimiento** (caché, carga diferida) o **añadir seguridad o registro** sin modificar el objeto real.

RESUMEN GENERAL

Patrón	Problema que resuelve	Ejemplo típico	Momento ideal de uso
Adapter	Conectar clases con interfaces incompatibles	Adaptador HDMI a USB	Cuando se integran sistemas o librerías incompatibles
Decorator	Agregar o quitar funcionalidades sin herencia	Ventana con bordes y sombras	Cuando se quiere extender comportamiento en tiempo de ejecución

Facade	Simplificar el acceso a un subsistema complejo	BancoFacade para operaciones financieras	Cuando se necesita una interfaz simple para un sistema complejo
Proxy	Controlar acceso o añadir lógica auxiliar	Proxy de caché o de seguridad	Cuando hay que controlar recursos o acceso al objeto real

COMPORTAMIENTO

PATRONES DE DISEÑO DE COMPORTAMIENTO

Estos patrones se enfocan en **cómo los objetos interactúan y se comunican** entre sí para distribuir responsabilidades y mantener el bajo acoplamiento del sistema.

1. Chain of Responsibility

Qué es:

Permite pasar solicitudes por una cadena de manejadores. Cada uno decide si la procesa o la pasa al siguiente.

Ejemplo:

Un sistema de soporte técnico: la solicitud pasa del *agente de primer nivel* → *supervisor* → *gerente*, según la complejidad.

Cuándo usarlo:

Cuando varias clases pueden manejar una solicitud y no se quiere acoplar el emisor con un receptor específico.

2. Command

Qué es:

Convierte una solicitud (acción) en un objeto, desacoplando al emisor del ejecutor y permitiendo operaciones como *deshacer* o *almacenar comandos*.

Ejemplo:

Un editor de texto con los comandos **Copiar**, **Pegar**, **Deshacer** implementados como objetos **Command**.

Cuándo usarlo:

Cuando quieres **encolar operaciones, ejecutar tareas remotas o implementar “deshacer/rehacer”**.

3. Iterator

Qué es:

Permite recorrer los elementos de una colección sin exponer su estructura interna.

Ejemplo:

Usar `for element in lista` en Python o `iterator.next()` en Java sin saber si la colección es una lista, pila o árbol.

Cuándo usarlo:

Cuando necesitas **recorrer colecciones complejas sin exponer su implementación interna**.

4. Mediator

Qué es:

Centraliza la comunicación entre objetos para evitar dependencias directas y reducir el acoplamiento.

Ejemplo:

En una app de chat, el *ChatRoom* (mediador) coordina los mensajes entre usuarios, que no se comunican directamente entre sí.

Cuándo usarlo:

Cuando hay **muchas interacciones entre objetos** que vuelven difícil el mantenimiento del código.

5. Observer

Qué es:

Define una relación *uno a muchos*: cuando un objeto cambia su estado, notifica automáticamente a sus suscriptores.

Ejemplo:

Una app de clima donde varios dispositivos (suscriptores) reciben actualizaciones del servidor meteorológico (publicador).

Cuándo usarlo:

Cuando **cambios en un objeto deben reflejarse automáticamente en otros**, sin acoplarlos directamente.

6. State

Qué es:

Permite que un objeto altere su comportamiento cuando cambia su estado interno, evitando condicionales complejas.

Ejemplo:

Una **CuentaBancaria** cambia su comportamiento entre los estados **Activa**, **Congelada** o **Cerrada**.

Cuándo usarlo:

Cuando el **comportamiento de un objeto depende de su estado actual** y se quiere eliminar estructuras **if-else** extensas.

RESUMEN GENERAL

Patrón	Problema que resuelve	Ejemplo típico	Momento ideal de uso
Chain of Responsibility	Varias clases pueden procesar una solicitud	Escalado en atención al cliente	Cuando no sabes quién procesará la solicitud
Command	Encapsular una acción como objeto	Comandos de un editor de texto	Cuando necesitas deshacer, ejecutar o almacenar acciones
Iterator	Recorrer colecciones sin exponer estructura	Recorre listas o árboles	Cuando hay colecciones complejas o privadas
Mediator	Reducir dependencias entre objetos	ChatRoom o panel GUI	Cuando hay comunicaciones cruzadas entre muchos objetos
Observer	Notificar a varios objetos ante cambios	App de clima, eventos UI	Cuando cambios deben propagarse automáticamente
State	Cambiar comportamiento según el estado	Cuenta bancaria o semáforo	Cuando hay múltiples estados con reglas diferentes

SISTEMAS WEB

PATRONES DE SISTEMAS WEB

Los patrones de sistemas web se dividen principalmente en dos grupos:

1. **Patrones de Presentación** (cómo se muestra la información al usuario).
2. **Patrones de Controlador** (cómo se gestionan las solicitudes y la lógica de flujo).

1. Patrones de Presentación

Template View

Qué es:

Una plantilla HTML que contiene *marcadores o etiquetas* (placeholders) que son reemplazados por datos dinámicos cuando la página se carga.

Ejemplo:

Un archivo `libro.html` con marcadores `${libro.titulo}` y `${autor.nombre}` reemplazados por datos del modelo cuando se genera la página.

Cuándo usarlo:

- Cuando el diseño y la lógica deben mantenerse **claramente separados**.
- Ideal para aplicaciones con **diseñadores y programadores trabajando en paralelo**.
- Ventaja: código de plantilla fácil de entender.
- Desventaja: riesgo de incluir lógica compleja dentro de la vista.

Transform View

Qué es:

Convierte los datos del dominio de negocio en HTML mediante una clase transformadora encargada solo del renderizado.

Ejemplo:

Un objeto `Transformer` con métodos como `transformarLibro()` y `transformarAutor()` que toman los datos del modelo y los transforman en HTML.

Cuándo usarlo:

- Cuando se quiere **generar HTML desde el backend** sin depender de un servidor web.
 - Útil para pruebas locales o plantillas simples.
 - Desventaja: cambiar la apariencia implica modificar varias clases transformadoras.
-

2. Patrones de Controlador

Page Controller

Qué es:

Cada página del sitio web tiene su propio controlador que maneja sus solicitudes y lógica específica.

Ejemplo:

Una página `login.jsp` podría tener un controlador `LoginController` con métodos `doGet()`, `doPost()` y `validarUsuario()`.

Cuándo usarlo:

- Ideal para **sitios pequeños o medianos** con controladores simples.
 - Ventaja: cada vista tiene su propia lógica separada.
 - Desventaja: en sistemas grandes hay **muchos controladores**, dificultando el mantenimiento.
-

Front Controller

Qué es:

Un único controlador central que gestiona todas las solicitudes del sitio web, delegando la ejecución a “comandos” específicos.

Ejemplo:

Un `Handler` recibe la solicitud, determina qué clase `Command` ejecutar (por ejemplo, `LoginCommand`, `RegistroCommand`), y cada comando elige la vista a mostrar.

Cuándo usarlo:

- Ideal para **aplicaciones grandes o con muchas páginas**, donde se necesita una lógica de control centralizada.
- Ventajas:
 - Fácil extender funcionalidades agregando nuevos comandos.
 - Permite aplicar decoradores (autenticación, internacionalización, etc.).
- Desventajas:
 - Riesgo de cuello de botella.
 - Mayor complejidad para mantener un controlador único.

RESUMEN GENERAL

Categoría	Patrón	Qué hace	Ejemplo	Momento ideal de uso
Presentación	Template View	Reemplaza marcadores en HTML con datos dinámicos	<code>\${usuario.nombre}</code> en HTML	Cuando se separa el diseño del código
Presentación	Transform View	Convierte datos de negocio a HTML mediante clases	<code>transformarLibro()</code>	Cuando se quiere probar sin servidor web
Controlador	Page Controller	Controlador por página	<code>LoginController</code> , <code>RegistroController</code>	Cuando cada vista tiene lógica simple
Controlador	Front Controller	Controlador único que maneja todas las solicitudes	<code>Handler + Comandos</code>	Cuando se necesita un flujo centralizado y extensible

Resumen final (Cuando aplicar cada uno)

Categoría	Patrón	Qué hace / Propósito	Ejemplo práctico	Momento ideal de uso
Creacional	Factory Method	Define una interfaz para crear objetos, dejando que las subclases decidan qué clase instanciar.	Sistema de notificaciones (SMS, Email).	Cuando hay múltiples variantes del mismo producto.
Creacional	Abstract Factory	Crea familias de objetos relacionados sin especificar sus clases concretas.	GUI multiplataforma: botones y menús para Windows/Mac.	Cuando los objetos deben ser coherentes entre sí.
Creacional	Builder	Construye objetos complejos paso a paso.	Construcción de casas, autos o documentos PDF.	Cuando un objeto requiere muchos pasos o configuraciones .
Creacional	Prototype	Crea nuevos objetos clonando instancias existentes.	Clonar enemigos en un videojuego.	Cuando crear desde cero es costoso.
Creacional	Singleton	Asegura que solo exista una instancia global.	Logger o conexión a base de datos.	Cuando se necesita un único punto de acceso a un recurso.
Estructural	Adapter	Permite que clases con interfaces incompatibles trabajen juntas.	Adaptador HDMI ↔ USB.	Cuando se integran sistemas o librerías incompatibles.

Estructural	Decorator	Agrega o elimina funcionalidades dinámicamente sin modificar la clase original.	Ventana con borde o sombra adicional.	Cuando se necesita extender comportamiento o en tiempo de ejecución.
Estructural	Facade	Ofrece una interfaz simple a un subsistema complejo.	BancoFacade que gestiona operaciones.	Cuando se quiere simplificar la interacción con sistemas complejos.
Estructural	Proxy	Controla el acceso a un objeto, añadiendo lógica (seguridad, caché, logs).	Proxy de caché o de autenticación.	Cuando se requiere control o acceso indirecto a un recurso.
Comportamiento	Chain of Responsibility	Envía solicitudes a través de una cadena de manejadores.	Escalado en soporte técnico (agente → supervisor → gerente).	Cuando múltiples objetos pueden manejar una solicitud.
Comportamiento	Command	Encapsula una acción o solicitud en un objeto.	Comandos "Copiar", "Pegar", "Deshacer" en un editor.	Cuando se necesitan operaciones reversibles o ejecutadas en cola.
Comportamiento	Iterator	Permite recorrer colecciones sin conocer su estructura interna.	Iterar sobre una lista o árbol.	Cuando se necesita recorrer colecciones complejas o privadas.
Comportamiento	Mediator	Centraliza la comunicación entre objetos.	ChatRoom que coordina usuarios.	Cuando hay muchas dependencias cruzadas entre objetos.

Comportamiento	Observer	Notifica automáticamente a los objetos suscriptores ante un cambio.	App de clima o sistema de alertas.	Cuando varios objetos deben reaccionar a un cambio.
Comportamiento	State	Cambia el comportamiento de un objeto según su estado interno.	Cuenta bancaria (activa, congelada, cerrada).	Cuando el objeto tiene varios estados con reglas distintas.
Sistemas Web	Template View	Usa marcadores en HTML reemplazados por datos dinámicos.	<code>\${usuario.nombre}</code> en HTML.	Cuando se separa el diseño de la lógica del sistema.
Sistemas Web	Transform View	Convierte datos del modelo en HTML mediante clases transformadoras.	Clase Transformer con métodos <code>transformarLibro()</code> .	Cuando se quiere renderizar HTML desde backend sin servidor web.
Sistemas Web	Page Controller	Cada página tiene su propio controlador.	LoginController, RegistroController.	Cuando cada vista tiene lógica simple e independiente.
Sistemas Web	Front Controller	Un único controlador gestiona todas las solicitudes.	Handler con comandos (LoginCommand, RegistroCommand).	Cuando se necesita un flujo centralizado y extensible.

Persistencia:



PATRONES DE HERENCIA DE CLASES

Estos patrones abordan cómo representar en una base de datos relacional una **jerarquía de herencia de clases** en un modelo orientado a objetos.

El documento presenta tres enfoques principales:



1. Single Table Inheritance



Qué hace:

Toda la jerarquía de herencia se representa en **una sola tabla** que contiene todos los atributos de las clases padre e hijas.



Ventajas:

- Solo una tabla que mantener.
- No requiere *joins* para consultas.
- Mover atributos entre clases no implica cambios estructurales.



Desventajas:

- Desperdicio de espacio (valores nulos).
 - Dificultad para entender el significado de columnas.
 - Dificultad al agregar nuevas clases hijas.
 - Posible impacto en rendimiento.
-



2. Class Table Inheritance



Qué hace:

Cada clase de la jerarquía tiene **su propia tabla**, vinculadas por su identificador.



Ventajas:

- No desperdicia espacio.
- Modelo más entendible y estructurado.
- Agregar nuevas subclases tiene menor impacto.

✗ Desventajas:

- Requiere *joins* complejos.
- Mover atributos entre clases cambia el esquema.
- La tabla de la clase padre puede ser un cuello de botella.



3. Concrete Table Inheritance



Qué hace:

Cada **clase concreta** (sin subclases hijas) tiene su propia tabla completa, incluyendo atributos heredados.



Ventajas:

- No hay espacio desperdiciado.
- Lectura más eficiente (no requiere *joins*).
- Cada tabla se consulta solo cuando se usa la clase correspondiente.
- Bajo impacto al agregar nuevas clases hijas.

✗ Desventajas:

- Cambios en atributos de la clase padre afectan varias tablas.
- Consultas sobre la clase base son más complejas.



CUADRO COMPARATIVO DE PATRONES DE HERENCIA

Patrón	Cómo mapea la herencia	Ventajas principales	Desventajas principales	Cuándo usarlo
--------	------------------------	----------------------	-------------------------	---------------

Single Table Inheritance	Una única tabla para toda la jerarquía (con campos para todos los atributos).	Simple de mantener, sin <i>joins</i> , fácil de consultar.	Espacio desperdiciado (valores nulos), difícil de escalar, columnas ambiguas.	Cuando el número de subclases es pequeño y las diferencias entre ellas son mínimas.
Class Table Inheritance	Una tabla por clase (padre e hijas enlazadas por ID).	Sin duplicación de datos, estructura clara y coherente.	Consultas más lentas por múltiples <i>joins</i> .	Cuando se necesita claridad en el modelo y se prefiere integridad normalizada.
Concrete Table Inheritance	Solo las clases concretas tienen su propia tabla (duplicando atributos heredados).	No usa <i>joins</i> , lectura rápida, bajo impacto al extender jerarquías.	Duplicación de datos, difícil mantener coherencia.	Cuando se prioriza la velocidad de lectura y cada subclase tiene independencia fuerte.

EJEMPLOS APLICADOS Y JUSTIFICADOS

Single Table Inheritance — Ejemplo: Sistema de Animales

Tabla: `Animal`

```
-----
id | nombre | pelaje | plumaje | raza | tipo
-----
1  | Rex    | corto  | NULL    | Pastor Alemán | Perro
2  | Loro   | NULL   | verde   | NULL         | Ave
```

Por qué usarlo:

Ideal si la jerarquía no cambia frecuentemente y se requieren consultas rápidas a una sola tabla, como en sistemas pequeños o con pocos tipos de entidades.

Class Table Inheritance — Ejemplo: Sistema de Vehículos

Tabla `Vehiculo`: `id`, `marca`

Tabla `Auto`: `id`, `puertas`, `tipo_motor`

Tabla `Camion`: `id`, `capacidad_carga`

Por qué usarlo:

Adecuado para aplicaciones empresariales donde la estructura del modelo debe reflejar fielmente las clases y sus relaciones, garantizando consistencia y normalización.



Concrete Table Inheritance — Ejemplo: Sistema Bancario

Tabla CuentaAhorros: id, saldo, tasa_interes

Tabla CuentaCorriente: id, saldo, limite_descubierto

Por qué usarlo:

Ideal cuando cada tipo concreto se maneja de forma independiente y la aplicación consulta solo una de las subclases a la vez, optimizando rendimiento en lectura.



Conclusión general

Los patrones de herencia no son excluyentes: **pueden combinarse** según el tipo de jerarquía o requerimientos del sistema.

Por ejemplo:

- **Single Table** para clases simples o poco diversas.
- **Class Table** para modelos empresariales estructurados.
- **Concrete Table** para rendimiento y simplicidad de lectura.

Interacción con BD

RESUMEN GENERAL – PATRONES DE INTERACCIÓN CON BASES DE DATOS

Estos patrones definen **cómo los objetos de una aplicación acceden y manipulan los datos almacenados en tablas** de una base de datos relacional, buscando desacoplar la lógica del negocio de la lógica de persistencia.

1 Active Record

Qué hace:

Cada objeto combina la **lógica del dominio** (atributos y comportamientos) y la **interacción con la base de datos** (consultas, inserciones y actualizaciones).

Características clave:

- Los métodos CRUD están en el mismo objeto.
- Los SQL están embebidos en la clase.
- Corresponde 1 objeto ↔ 1 tabla.

Ventajas:

- ✓ Sencillo de implementar.
- ✓ Eficaz cuando el modelo es simple.

Desventajas:

- ✗ Fuerte acoplamiento entre lógica y datos.
 - ✗ Difícil manejar herencias o colecciones complejas.
 - ✗ La clase “conoce” directamente la estructura de la base de datos.
-

2 Table Data Gateway

Qué hace:

Un **objeto intermediario (Gateway)** gestiona las operaciones sobre una tabla específica.

Características:

- Cada Gateway representa una tabla.
- Encapsula las consultas SQL y operaciones CRUD.
- Las entidades delegan la interacción a este Gateway.

Ventajas:

- ✓ Separa la lógica de negocio de la persistencia.
- ✓ Puede combinarse con procedimientos almacenados.

Desventajas:

- ✗ Dependencia circular (la entidad sabe que existe el Gateway).
 - ✗ Ligera duplicación de lógica entre capas.
-

3 Row Data Gateway

Qué hace:

Cada fila de la tabla se representa como un **objeto Gateway**, y un objeto adicional (Finder) realiza las búsquedas.

Características:

- Cada Gateway representa una fila.
- Los SQL residen dentro del Gateway.
- Usa un Finder para obtener listas o entidades específicas.

Ventajas:

- ✓ Encapsula completamente la lógica de acceso.
- ✓ Desacopla estructura de entidad y tabla.
- ✓ Puede usarse con procedimientos almacenados.

Desventajas:

- ✗ El objeto entidad conoce la existencia del repositorio.
 - ✗ Mayor complejidad al mantener entidad, Finder y Gateway.
-

4 Data Mapper

Qué hace:

Un objeto **Mapper** se encarga de transferir datos entre las entidades del dominio y

las tablas, **sin que las entidades conozcan la base de datos.**

Características:

- Total independencia entre modelo y persistencia.
- Los SQL están dentro del Mapper.
- Soporta herencia, colecciones y relaciones complejas.

Ventajas:

- ✓ Ideal para sistemas grandes.
- ✓ Permite estructuras complejas y reutilizables.
- ✓ Total desacoplamiento de la capa de negocio.

Desventajas:

- ✗ Mayor complejidad.
 - ✗ Requiere frameworks o librerías ORM (como Hibernate o Entity Framework).
-

5 Dependent Mapping

Qué hace:

Un objeto “dueño” gestiona la persistencia de otro objeto “dependiente”.

Ejemplo:

Un **Álbum** guarda sus **Pistas** —el mapeo de Pista depende del de Álbum.

Ventajas:

- ✓ Simplifica relaciones “uno a muchos” contenidas.
- ✓ Evita inconsistencias al eliminar o modificar.

Desventajas:

- ✗ No reutilizable: el dependiente no puede existir fuera del dueño.
-

6 Objeto Grande (LOB) Serializado

Qué hace:

Serializa un objeto o colección en un único campo (BLOB o CLOB).

- **BLOB:** formato binario (rápido, compacto).
- **CLOB:** formato texto (JSON/XML).

Ventajas:

- ✓ Simplifica el guardado de estructuras complejas.
- ✓ Permite almacenar “snapshots”.

Desventajas:

- ✗ No permite consultas SQL dentro del objeto.
 - ✗ Cambios de formato pueden romper la compatibilidad.
 - ✗ Riesgo de duplicar datos.
-

7 Data Transfer Object (DTO)

Qué hace:

Objeto que **transporta datos entre capas o sistemas**, reduciendo el número de llamadas remotas.

Características:

- Transfiere información serializada.
- Ideal para arquitecturas cliente-servidor o distribuidas.
- Usa “Assembler” para convertir entre entidad ↔ DTO.

Ventajas:

- ✓ Reduce las llamadas a la red o API.
- ✓ Facilita el transporte de múltiples atributos.
- ✓ Reutilizable y serializable.

Desventajas:

- ✗ Redundancia de clases.
 - ✗ Debe mantenerse sincronizado con la entidad.
 - ✗ Requiere lógica de ensamblado.
-



CUADRO COMPARATIVO: POR QUÉ Y CUÁNDO USAR CADA PATRÓN

Patrón	Por qué usarlo	Cuándo aplicarlo	Nivel de acoplamiento	Complejidad
--------	----------------	------------------	-----------------------	-------------

Active Record	Simplicidad total y correspondencia directa objeto-tabla.	En sistemas pequeños o con modelos muy simples.	Alto	Baja
Table Data Gateway	Centraliza la lógica SQL por tabla, sin mezclarla con la entidad.	En sistemas medianos donde se busca encapsular acceso por tabla.	Medio	Media
Row Data Gateway	Permite controlar cada fila como objeto individual.	Cuando se requiere un control detallado por registro.	Medio	Media
Data Mapper	Desacopla completamente la lógica de negocio del almacenamiento.	En proyectos grandes o con modelos complejos (usando ORM).	Bajo	Alta
Dependent Mapping	Gestiona relaciones “uno a muchos” sin crear inconsistencias.	Cuando una clase solo tiene sentido dentro de otra (composición).	Bajo	Media
Objeto LOB Serializado	Guarda estructuras complejas fácilmente (sin diseñar tablas adicionales).	Cuando no se requiere consultar internamente los objetos.	Bajo	Media
Data Transfer Object (DTO)	Optimiza rendimiento en comunicación entre capas o sistemas.	En arquitecturas distribuidas o APIs con gran volumen de datos.	Bajo	Media

Anti Patrones

Estructura del ciclo de un antipatrón

Según la *línea de tiempo* mostrada en la página 7 del documento:

1. **Problema inicial:** Se detecta una necesidad.
 2. **Antipatrón:** Se aplica una “solución rápida” (parece resolverlo).
 3. **Nuevo problema:** Aparecen fallas o limitaciones por esa decisión.
 4. **Refactoring:** Se reestructura el código o arquitectura para corregir los efectos negativos.
-

ANTIPATRONES MÁS COMUNES EN DESARROLLO

1. *Spaghetti Code*

Descripción: Código desordenado, sin estructura ni documentación, difícil de mantener.

Causas: Desconocimiento, flojera o falta de diseño previo.

Síntomas:

- Muchos métodos sin parámetros.
- Variables globales innecesarias.
- Poca relación entre clases.
- No se usa herencia ni polimorfismo.
- Dificultad para reutilizar el código.

Frase típica: “Es más fácil reescribirlo que entenderlo.”

2. *The Blob*

Descripción: Una sola clase concentra demasiadas responsabilidades que deberían distribuirse en otras.

Causas: Prisa o pereza en el diseño.

Síntomas:

- Clases con más de 60 métodos o atributos.
- Atributos no relacionados.
- Falta de orientación a objetos.
- Relación mínima entre clases.

Frase típica: “Esta clase es el corazón de toda la arquitectura.”

3. *Cut & Paste*

Descripción: “Reutilización” de código mediante copiar y pegar fragmentos.

Causas: Pereza, desconocimiento de refactorización o reutilización adecuada.

Síntomas:

- Mismo error aparece en distintas partes del código.
- Alto costo de mantenimiento.

Frase típica: “¿No habíamos corregido ya ese error?”

4. *Golden Hammer*

Descripción: Construir toda una solución basándose en una sola herramienta o tecnología.

Causas: Ignorancia, orgullo, mente cerrada.

Síntomas:

- Problemas de escalabilidad y desempeño.
- Arquitectura definida por un solo producto.
- Requerimientos adaptados a la herramienta.
- Conocimiento muy limitado del equipo.

Frase típica: “Nuestra base de datos es nuestra arquitectura.”

5. *Lava Flow*

Descripción: Código legado sin documentación ni mantenibilidad, que sigue fluyendo en producción.

Causas: Avaricia, envidia, flojera o falta de transferencia de conocimiento.

Síntomas:

- Estructuras o funciones complejas sin explicación.
- Variables o fragmentos de código sin propósito.
- Comentarios obsoletos o sin justificación.
- Sistemas antiguos imposibles de modificar.

Frase típica: “No toques esa parte, nadie sabe cómo funciona.”

6. *Obsolescencia Continua*

Descripción: Falta de actualización de librerías o tecnologías usadas en el sistema.

Causas: Entorno cambiante o falta de planificación de mantenimiento.

Síntomas:

- Uso de estándares cerrados u obsoletos.
- Aplicaciones que no se actualizan hace mucho tiempo.

Frase típica: “¿De verdad hay que actualizar las librerías?”

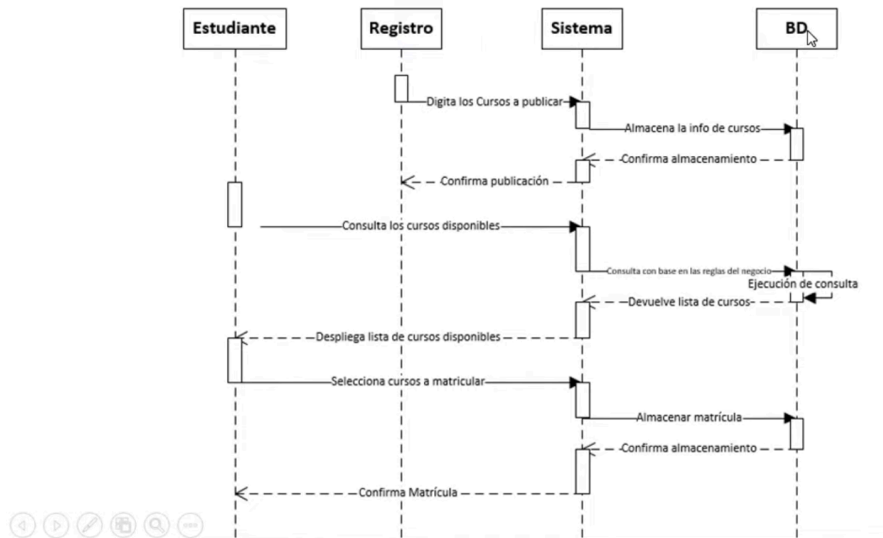
CONCLUSIÓN GENERAL

Los **antipatrones** aparecen cuando se prioriza la **velocidad de desarrollo sobre la calidad** o cuando no se aplican principios de diseño adecuados.

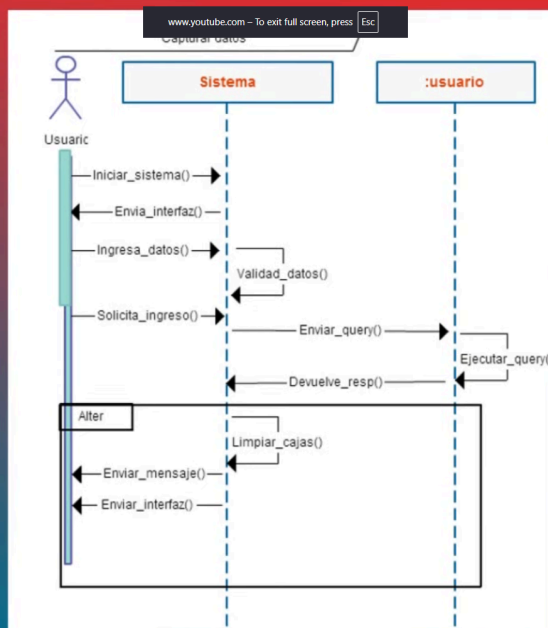
Su identificación temprana permite aplicar **refactoring**, es decir, reestructurar el código para recuperar la claridad, cohesión y mantenibilidad.

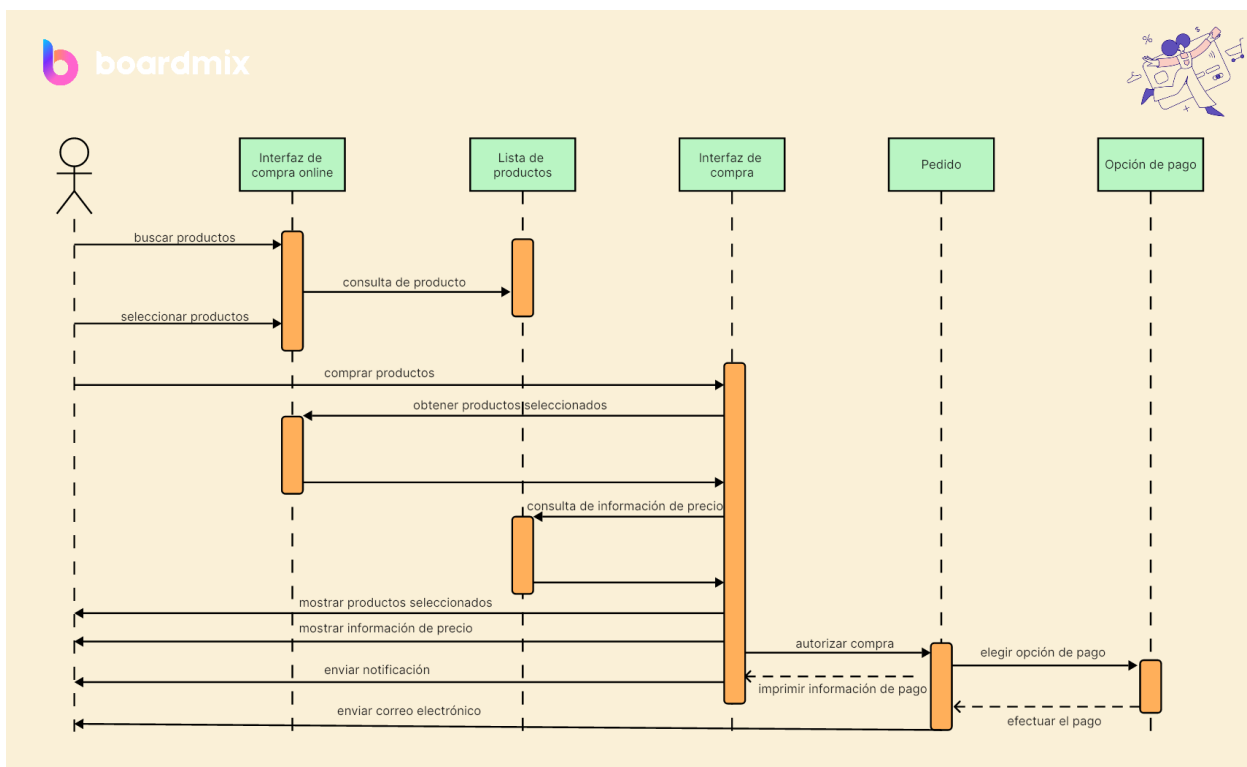
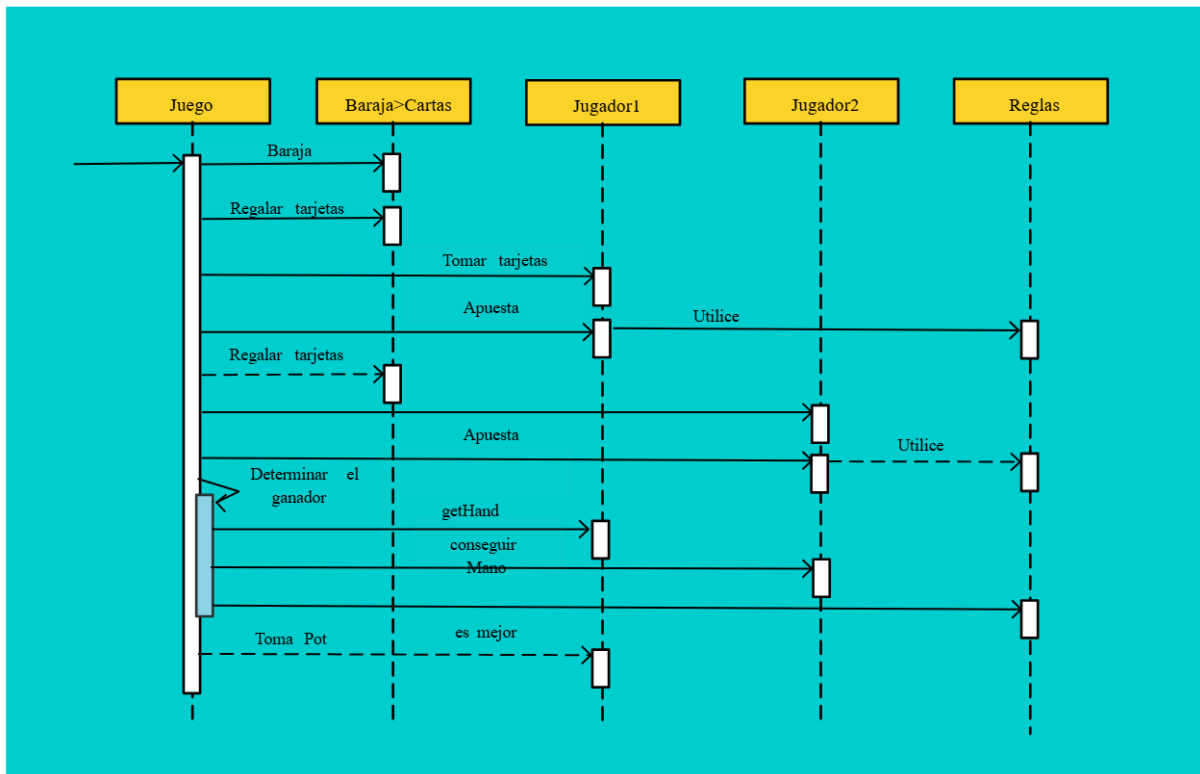
DIAGRAMAS DE INTERACCIÓN - Ejemplos prácticos de uso. (al final pa imprimir)

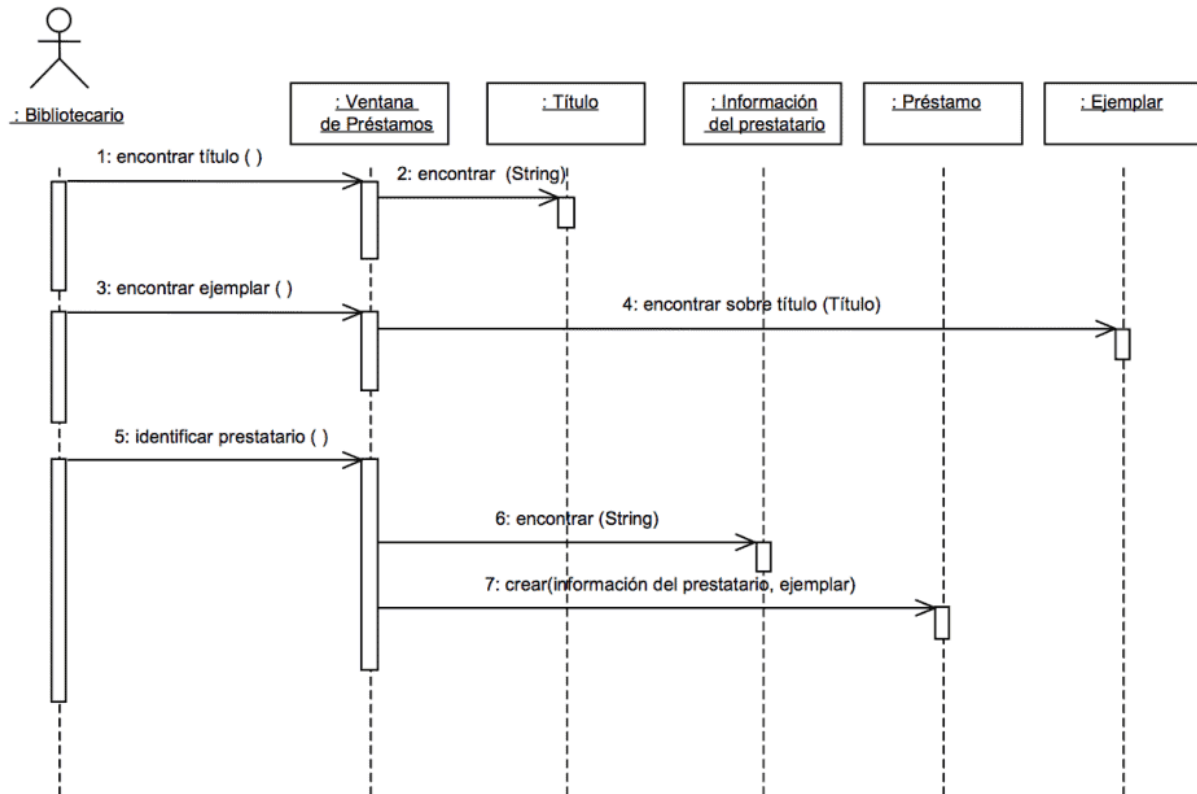
EJEMPLO



SOLUCIÓN







Ejemplo de diagrama de secuencia

